jHWaves
jHWavesC

Coastal-trapped wave modal calculations for stratified conditions using Julia

K.H. Brink

April 16, 2026

0. Overview:

This document describes two Julia software packages (each set of functions contained in its own single ".jl" file) for computing coastal trapped wave functions in a stratified ocean with a sloping bottom. Both packages allow for a mean alongshore flow. More context on these waves can be found in Brink (2023). The two packages are very similar, but one (jHWaves) is for a real wave frequency, while the other (jHWavesC) is for a complex frequency.

1. Introduction:

There exists a set of publicly available Matlab mfiles that can be used to calculate coastal-trapped wave properties in a range of conditions and geometries (https://www2.whoi.edu/staff/kbrink/software/matlab-links/). While these are free for anyone to use, they are only viable for people having access to the Matlab system, which involves paying for a license. Thus, I have been advised that the fair thing to do is to translate these codes so that they can be used in a framework which is freely available to anyone. After some searching around, experimentation and advice from colleagues, I have settled on the Julia system (https://julialang.org/learning/). This well-regarded system has many of the same qualities as Matlab, and is relatively easy to learn, knowing Matlab.

There are, of course, some differences from the Matlab system. One is that the core Matlab system already includes some software packages (such as for interpolation, graphics, and searching), a considerable convenience. With Julia, many of these capabilities require the activation of additional free software packages. This is really only a minor inconvenience once one installs the appropriate package. Further, there are a number of other aspects where the two systems vary (https://docs.julialang.org/en/v1/manual/noteworthy-differences/), some of which take some getting used to.

There is at least one other coastal-trapped wave solver that is readily available: that of Furue and Tanaka (2025: https://eartharxiv.org/repository/view/10924/ , https://github.com/ryofurue/ctwmodes). Their remarkable code (written in a combination of

Julia and Fortran) computes all of the coastal trapped wave modes for a given configuration in one shot. However, it has some restrictive aspects, including that 1) it only computes modes in the coastal long wave approximation (so that it can not compute a dispersion curve), 2) It does not allow for a mean alongshore flow, nor for bottom friction. 3) It is limited only to real wave frequencies/phase speeds. Further, the Furue and Tanaka formulation uses *z* coordinates, resulting in a steppy bottom, while the present code uses a terrain-following (”sigma”) coordinate system. Depending on the application, a user might find one or the other of these coordinate systems preferable. The present code is written entirely in Julia, so that one does not need to deal with the Fortran interface.

2. Using the Julia-based system:

After installing Julia on your computer, you will need to install a number of software packages in order to use this code. This needs to be done only once, but the packages need to be activated (with the “using” command) any time you want to use them. For the stratified wave code (in file jHWaves.jl), one needs the following packages:

CairoMakie
GLMakie
Interpolations
JLD2
Makie
NLopt
Printf
SparseArrays.

Plus the “Revise” package is also handy when writing and revising code. One adds a package simply as follows:

```
julia> using PkG
julia> Pkg.add(“CairoMakie”)  .
```

In order to use the code in the files, one needs first to establish a “module” which defines the working space that you will use. The module file (ModHW.jl) that works for the stratified wave code contains simply the following lines:

```
module ModBW

    using Revise
    using Makie
    using CairoMakie
    using JLD2
    using Interpolations
    using NLopt
    using Printf
```

```
        using SparseArrays

        # println("You may want to change directories”)

        cd("C:\\Projects\\jwaves\\jHWaves")

    end
```

(The # marker denotes a comment). This file defines the module/working space, activates the packages that will be needed, and sets the directory to where the stratified wave (“HW” for Hybrid Wave) code is to be found. The user will likely have a different path for this.

A typical session might begin as follows. First a module is established by using the “include” command. This activates the above code, and places you in the desired directory. Note that for this purpose, you have to use double slashes, not single ones. Next, you include the baroclinic wave code, again with an “include” command, but now, you do not need to state the whole path, because you are in the desired directory. Next, load (or create: see below) the tuple (an array of various arrays) that gives the wave code its needed information (more on this below). For a given wave problem, the code files (like jHWaves.jl) contains all of the individual functions you need to proceed. Finally, you can run the code by calling the central function jHWavesM. Thus (I have placed in red what Julia provides, in blue what you, as a user type in, and in black what the program outputs):

```
               _
   _       _ _(_)_     |  Documentation: https://docs.julialang.org
  (_)     | (_) (_)    |
   _ _   _| |_  __ _   |  Type "?" for help, "]?" for Pkg help.
  | | | | | |/ _` | |
  | | |_| | | (_| |  |  Version 1.12.0 (2025-10-07)
 _/ |\__'_|_|_|\__'_|  |  Official https://julialang.org release
|__/                   |

julia> include("C:\\Projects\\jwaves\\jHWaves\\ModHW.jl")
Main.ModBW

julia> include("jHWaves.jl")
inputf (generic function with 1 method)

julia> testb = load_object("testb.jld2")
(100, 30, 2.0e-6, 1, 0, 1, 1, 0.0001, 100.0, 0.001, 1, 1.0e-8, 1.0e-8, 4, [0.0 40.0 50.0 100.0], [10.0
100.0 500.0 500.0], 2, [0.0 100.0], [0.05 0.05], 2, 1000.0, 1.0, [1.0e-5 1.0e-5], 30.0, 40.0, 0.0,
200.0, 200.0, 20.0, 20.0, 1, 0)

julia> jHWavesM(testb)
nn, mm = 100,  30
```

Free surface
Closed BC at x = 0
Open BC at x = xmax

f = 1.000e-04 rad/sec

Max consistency ratio = 6.545 at x = 41.24 km
This should be kept less than one, and definitely less than 10

fs/f becomes negative somewhere: symmetric instability is possible
f* X 10^5 is contoured in red along with density

Wavenumber = 1.000e-08 rad/cm

Iterations:
wr, rrr = 2.0000e-06, 1.3773e-26
wr, rrr = 2.0500e-06, 1.2154e-26
wr, rrr = 2.1000e-06, 1.0573e-26
wr, rrr = 2.1500e-06, 9.0363e-27
wr, rrr = 2.2500e-06, 6.1230e-27
wr, rrr = 2.3500e-06, 3.5217e-27
wr, rrr = 2.5500e-06, 1.7277e-28
wr, rrr = 2.7500e-06, 2.2414e-27
wr, rrr = 2.7500e-06, 2.2414e-27
wr, rrr = 2.6500e-06, 2.2998e-28
wr, rrr = 2.4500e-06, 1.4305e-27
wr, rrr = 2.6000e-06, 2.8434e-31
wr, rrr = 2.6500e-06, 2.2998e-28
wr, rrr = 2.5750e-06, 4.1248e-29
wr, rrr = 2.6250e-06, 5.9720e-29
wr, rrr = 2.5875e-06, 8.8460e-30
wr, rrr = 2.6125e-06, 1.6814e-29
wr, rrr = 2.5937e-06, 1.5078e-30
wr, rrr = 2.6062e-06, 5.3330e-30
wr, rrr = 2.5969e-06, 1.2195e-31
wr, rrr = 2.5937e-06, 1.5078e-30
wr, rrr = 2.5984e-06, 8.3807e-33

Converged!

wr, rrr = 2.5984e-06, 8.3807e-33

Perturbation frictional decay time = 9.1513e+04 sec

fs/f becomes negative somewhere: symmetric instability is possible
f* X 10^5 is contoured in red along with density

EKE = 1.7621e-03, Interior EPE = 2.6133e-05
Surface EPE = 1.3739e-05
Do you want to save these results? 0 (no) or 1 (yes) 1
Enter the name of the desired file (.jld2 will be appended to this)
BaroSample

Saved as a tuple in file BaroSample.jld2
Contents of the tuple:

1) f (Coriolis parameter: 1/sec)
2) ilong (= 0 for long wave, = 1 for general)
3) icbc (= 0 for coastal wall, = 1 for open x = 0 BC)
4) iobc (= 0 for wall at offshore boundary, = 1 for open)
5) del (= 0 for rigid lid, = 1 for free surface)
6) rll (wavenumber)
7) wnn (real frequency and perturbation damping rate)
8) xgr (x grid in km)
9) zgr (z grid in m)
10) p (pressure)
11) xr (x for bottom frictional coefficient: km)
12) rr (bottom frictional coefficient: cm/sec)
13) vzero (extreme value of mean velocity: cm/sec)

You can load this tuple with the 'load_object' command.

julia>

This example includes a simple running of the basic baroclinic wave code, and shows how, once a wave mode is detected, you have the option of saving the results to a file that you can name. This function also leads to plots of the modal structure, dispersion curve (if more than one wavenumber is called for) and other items of interest. You do not see any of the needed "using" commands because they have been incorporated into the modHW.jl file.

In order to create the needed tuple, use the function (for this case) jHWavesSetup (within file jHWaves.jl). You call the function, and it asks you a sequence of questions that will build up the tuple testb:

julia> testb = jHWavesSetup()

Coastal-Trapped waves with real frequency

This function will ask you a sequence of questions
that are used to build the input tuple.

How many total gridpoints do you want in the cross shelf direction? (nn) 100
How many total gridpoints do you want in the vertical? (mm) 30
First guess at frequency (rad/sec)? 2e-6
Enter 0 for a rigid lid, 1 for a free surface (del) 1
Enter 0 for a closed x= 0 boundary, 1 for open (icbc) 0
Enter 0 for a closed x =xmax boundary, 1 for an open (iobc) 1
Enter 1 for general frequency, wavenumber or 0 for long wave limit 1
Enter the Coriolis parameter (f) (rad/sec) 1e-4
Enter the domain width (xmax) (km) 100
Enter the nominal fractional accuracy for the frequency solution (eps) .001
Enter the number of frequencies to be computed (npts) 1
Enter the first alongshore wavenumber to use (rlz) (rad/cm) 1e-8
How many distance, depth pairs will you provide (ndep >=1) 4
Array of offshore distances for depth values (xdep in km) (dimension ndep) [0 40 50 100]
Array of depths corresponding to xdep (depr in m) [10 100 500 500]
Number of distance, bottom friction pairs to read (nr) 2
Offshore distances for bottom friction values (xr in km) [0 100]
Array of bottom friction values corresponding to xr (rr in cm/sec) [0.05 0.05]
Depth increment for input of Nsquared values? (zr in m) 1000
Exponential tail length for Nsquared extrapolation (alph in km) 1
Nsquared values starting at the surface (nsqr in rad^2/sec^2) (nnsq values) [1e-5 1e-5]
Input peak value of mean alongshore flow (vzero: cm/sec) 30
Input distance offshore to peak mean flow (km) 40
Input depth of peak mean flow (m) 0
Downward exponential scale of mean flow? (m) 200
Upward exponential scale of mean flow? (m) 200
Offshore exponential scale of mean flow? (km) 20
Onshore exponential scale of mean flow? (km) 20
Enter 1 for undisturbed Nsquared offshore, 0 for onshore 1
Enter 0 to skip pauses to see graphics, 1 to see graphics during execution 0
(100, 30, 2.0e-6, 1, 0, 1, 1, 0.0001, 100.0, 0.001, 1, 1.0e-8, 1.0e-8, 4, [0.0 40.0 50.0 100.0], [10.0 100.0 500.0 500.0], 2, [0.0 100.0], [0.05 0.05], 2, 1000.0, 1.0, [1.0e-5 1.0e-5], 30.0, 40.0, 0.0, 200.0, 200.0, 20.0, 20.0, 1, 0)

julia>

Note that this displays the completed tuple at the end. If you do not want to see it, just put a semicolon after the () when you call the function.

You can save this tuple to the current directory with the command:

```
julia> save_object("testb.jld2", testb)
```

The above actions create the tuple testb, display it, and then save it to a file named testb.jld2 . The "Setup" function also makes a single figure showing depth, mean flow and the bottom friction parameter.

Finally, you can modify an input tuple by using the function "jHWavesFinch":

```
julia> tnew = jHWavesFinch(testb);

    Function to create modified input tuple

First you need to select what you want to change.

    Options are:
      g:  Grid size
      w:  Initial Frequency guess
      c:  Coastal long wave limit
      b:  Boundary conditions
      f:  Coriolis parameter
      x:  Domain size
      d:  Dispersion curve definition
      e:  Nominal accuracy
      h:  Depth profile
      r:  Bottom friction
      n:  Stratification
      v:  Mean flow field
      p:  Pauses during execution

Any arrays are row arrays, not column arrays

Please type in the appropriate letter and hit return
h

Old number of input points = 4
 Enter number of (x, h) pairs  (> 0) 3
Old array of x locations for depth (km) = [0.0 40.0 50.0 100.0]
 Enter array of x locations where depth will be given (km) [0 40 50]
Old array of depths (m) = [10.0 100.0 500.0 500.0]
 Enter array of depths at locations given (m) [10 100 1000]

julia>
```

Note that the blue/red/black color convention still holds. Note, too, that when you provide depth (or friction coefficient r) values, you do not have to give values extending all the way to the offshore boundary. The code will just fill out the rest of the topography with the last value given (1000 m in this case).

More explanations are given in the following sections, but this section was mainly meant to show the basics of using this code in Julia.

3. The physical problem:

We seek to find the free wave solutions for linearized coastal-trapped waves along a straight coast. The depth $h(x)$ is uniform in the alongshore (y) direction. The offshore direction is x ($x = 0$ is the coast) and the vertical direction is z ($z = -h(x)$ is the bottom), where the origin is at the surface at the coast. Dissipative effects are limited to the infinitesimally thin bottom boundary, and (for jHWaves) are only included for calculating coefficients for use with the first order wave equation and (only when $v_0 = 0$) for estimating the perturbation frictional time scale T_f (Brink, 1990). For jHWavesC, where a complex frequency is allowed, frictional effects enter when $r \neq 0$ though the bottom and lateral boundary layers.

The equations of motion (e.g., Brink, 2006) are:

$$v_t + u\, v_{0x} + v_0 v_y + w v_{0z} + fu = -\rho_0^{-1} p_y + \rho_0^{-1} \tau^y{}_z \qquad (1a)$$

$$\delta u_t + \delta v_0 u_y - fv = -\rho_0^{-1} p_x + \delta \rho_0^{-1} \tau^x{}_z \qquad (1b)$$

$$0 = -p_z - g\rho_2 \qquad (1c)$$

$$u_x + v_y + w_z = 0 \qquad (1d)$$

$$\rho_{2t} + u\, \rho_{1x} + v_0\, \rho_{2y} + w\, \rho_{1z} = 0 \qquad (1e)$$

where u, v, w are the perturbation velocity components in the x, y, z directions, respectively, p is perturbation pressure, f is the Coriolis parameter, density is broken up as: $\rho_0 + \rho_1(x, z) + \rho_2(x, y, z, t)$ and $\rho_0 >> \rho_1 >> \rho_2$. Using $\delta = 1$ corresponds to a general frequency and wavenumber, while $\delta = 0$ produces the coastal long wave limit, and only allows one frequency to be computed.

In all cases, subscripted independent variables represent partial differentiation. The mean velocity $v_0(x, z)$ is in geostrophic balance with the mean pressure p_0 and density $\rho_0 + \rho_1$. Turbulent stresses in the x, y directions are given by τ^x , τ^y, and the bottom stress (τ_0^x, τ_0^y) is related to the interior bottom velocity by

$$\tau_0^x = \rho r\, u_B, \qquad \tau_0^y = \rho\, r\, v_B, \qquad (2a, 2b)$$

where $r(x)$ is a bottom resistance coefficient having units of velocity, and a subscript "B" denotes a quantity evaluated at the bottom. For jHWaves, the turbulent stresses are not used for

computing the modal structures, but they are used to compute the damping coefficients after the inviscid wave mode is known. However, for jHWavesC, which allows complex frequency, the bottom friction does affect the modal structure. We also define

$$N^2 = -g\rho_0^{-1}\,\rho_{1z} \qquad \text{and} \tag{3a}$$

$$M^2 = -g\rho_0^{-1}\,\rho_{1x}. \tag{3b}$$

The problem is solved by assuming that

$$p = p'(x, z)\,\exp[i(\omega t + ly)],\ \text{etc.}, \tag{4}$$

where the frequency ω and the alongshore wavenumber l are real. The "primes" are dropped henceforth. Using this assumption, the problem can be reduced to a single partial differential equation for pressure

$$\begin{aligned} 0 = {} & \omega' p_{xx} - 2\,\omega' s\, p_{xz} + \omega' N^{-2}(ff' - \delta\,\omega'^2)\, p_{zz} - \omega'(Q + s_z)\, p_x \\ & - [-\omega' sQ + (\omega' s)_x + lf^{-1} s(f^2 - \delta\,\omega'^2) - (\omega' N^{-2})_z(ff' - \delta\,\omega'^2) - (\omega' N^{-2})(sM^2)_z]p_z \\ & - [lfQ + \delta\,\omega' l^2 + lfs_z]p, \end{aligned} \tag{5}$$

where

$$\omega' = \omega + lv_0 \tag{6a}$$

$$s = M^2/N^2, \tag{6b}$$

$$f' = f + v_{0x} \tag{6c}$$

$$Q = (ff^* - \delta\,\omega'^2)^{-1}[(ff^* - \delta\omega'^2)_x - s(ff^* - \delta\,\omega'^2)_z] \tag{6d}$$

$$f^* = f + v_{0x} - M^2 s/f. \tag{6e}$$

This equation and the notation are similar to those of Mooers (1975 a, b), except that these are for hydrostatic conditions.

As noted by Brink (1981) and Dale et al. (2001), this equation admits a spurious solution at $\omega' = f$ when $v_{0z} = v_{0x} = 0$. The presence of this mode also distorts modal structures and dispersion curves for nearby frequencies. Typically, this spurious pressure mode is vertically uniform and it decays only slowly with distance offshore.

The boundary conditions are as follows:

z = 0:

$$w = \gamma\, (g\rho_0)^{-1}[p_t + v_0 p_y], \tag{7}$$

where $\gamma = 1$ for a free surface condition and $\gamma = 0$ for a rigid lid condition. In terms of pressure,

$$0 = (\omega'/N^2)(ff' - \delta\omega'^2)\, p_z \;- s\, \omega'\, p_x - [lfs - \omega'\gamma\, (ff^* - \delta\omega'^2)/g])\, p \tag{8}$$

z = - h(x):

$$0 = w + h_x u + U_{Ex} + V_{Ey}, \tag{9}$$

where the Ekman transport is given as

$$U_E = r(ff^* - \delta\omega'^{\,2})^{-1}(-i\,\omega'\,\delta u - fv)_B, \tag{10a}$$

$$V_E = \delta r(ff^* - \delta\omega'^{\,2})^{-1}\,(f^*\,u - i\,\omega'\,v)_B, \tag{10b}$$

and the subscript "B" denotes a quantity evaluated at the bottom (along $z = h(x)$). In jHWaves, This complete form is used only in computing the long wave frictional coefficients once the inviscid mode is known. In jHWavesC, this complete bottom boundary form is used at lowest order. For computing the inviscid modal structure, the boundary condition is:

$$0 = w + h_x u \tag{11}$$

In terms of pressure,

$$\begin{aligned} 0 = \quad & + [is\omega' - i\omega' h_x\ ]\, p_{xB} \\ & + \{ -i\,\omega'\,N^{-2}\,(ff' - \delta\,\omega'^2) + i\,s\,\omega'\,h_x\ \}\, p_{zB} \\ & + \{ islf\ - ilfh_x \}\, p. \end{aligned} \tag{12}$$

In jHWaves, r is taken to be zero for the purposes of computing a wave frequency, but then (if $v_0 = 0$), it will go back and compute a perturbation estimate for the frictional decay time. jHWavesC uses the complete bottom boundary condition as given in (9).

x = 0:

There is a choice of two boundary conditions at the coast.

Closed boundary condition:

This boundary condition is used when icbc = 0:

$$0 = u. \tag{13}$$

Since there is no bottom friction, this boundary condition is sensible regardless of the coastal wall height. Damping coefficient calculations do account for Ekman transport here.

Stated in terms of pressure, this inviscid condition is:

$$0 = - i\, \omega' h\, p_x - ilfh\, p + i\, \omega'\, sh\, p_z. \tag{14}$$

This is the form of the boundary condition used in jHWaves, the real-frequency formulation. A version that includes friction is then used for estimating the perturbation damping time, T_F.

In jHWavesC (the complex frequency form), a more complete boundary condition, that accounts for the bottom Ekman transport, is applied. This formulation assumes that the water is shallow enough that u does not vary substantially over the water depth at the wall. This condition will hold if $Nh(0)/fX \ll 1$, where X is a representative horizontal length scale (a scaling based on the vorticity equation).

Stated in terms of pressure, the jHWavesC boundary condition is:

$$0 = - i\, \omega' h\, p_x - ilfh\, p + i\, \omega'\, sh\, p_z \tag{15a}$$

$$+ R\, r(ff^* - \omega'^2)^{-1}\, [-(ff^* + \omega'^2)\, p_x + 2\, s(\omega')^2\, p_z - 2\, \omega' fl\, p]_B$$

where

$$R = (ff^* - \omega'^2)/(ff^* - \omega'^2)_B. \tag{15b}$$

“Open” boundary condition:

To mimic an open boundary (icbc = 1), we use

$$u_x = 0, \tag{16}$$

which does a reasonable job for linear coastal-trapped waves at an "open" offshore boundary, where variations should have decayed (see Brink, 1982). To help make sure this is used reasonably, this condition requires that the bottom be flat at $x = 0$.

Stated in terms of pressure, this condition is:

$$0 = -\omega' p_{xx} + s\omega' p_{xz} + [-lf' + \omega' (ff^* - \omega'^2)_x (ff^* - \delta\omega'^2)^{-1}] p_x \tag{17}$$

$$+ [lsv_{0x} + \omega' s_x - s\omega'(ff^* - \delta\omega'^2)_x (ff^* - \delta\omega'^2)^{-1}] p_z$$

$$+ lf (ff^* - \delta\omega'^2)_x (ff^* - \delta\omega'^2)^{-1} p$$

This boundary condition can only be expected to work well at subinertial frequencies. At higher frequencies, it can lead to internal wave reflection, rather than the free exit of internal waves.

x = L:

"Open" Boundary Condition

The offshore open boundary condition is the same as the coastal "open" position, and is called by setting iobc = 1. This seems to work well as long as the grid boundary ($x = L$) occurs far offshore of where the modal solutions have substantial structure. Further, problems can occur at superinertial frequencies. The condition, as used, requires that the bottom is flat at the offshore boundary. The condition is:

$$u_x = 0, \tag{18}$$

or, in terms of pressure:

$$0 = -\omega' p_{xx} + s\omega' p_{xz} + [-lf' + \omega' (ff^* - \delta\omega'^2)_x (ff^* - \delta\omega'^2)^{-1}] p_x \tag{19}$$

$$+ [lsv_{0x} + \omega' s_x - s\omega'(ff^* - \delta\omega'^2)_x (ff^* - \delta\omega'^2)^{-1}] p_z$$

$$+ lf (ff^* - \delta\omega'^2)_x (ff^* - \delta\omega'^2)^{-1} p.$$

Closed boundary condition

In addition, using iobc = 0 allows use of a closed offshore boundary condition (the same as for the coastal boundary): equations 14 or 15, depending on whether jHWaves or jHWavesC is applied.

At this point, the problem is converted into stretched coordinates that map the problem into a rectangular domain (e.g. Wang and Mooers 1976). Specifically,

$$\theta = z/h(x) \tag{20a}$$

and

$$x' = x. \tag{20b}$$

The problem is then solved numerically via resonance iteration. An arbitrary forcing is applied through the surface boundary condition (essentially, a localized wind stress curl near the grid center), and ω is searched to find a real (for jHWaves) or complex (for jHWavesC) frequency corresponding to a given wavenumber l. This search makes use of the Julia function "NLOpts.optimize" which is part of the NLopt software package.

4. More on the software:

There are two files/ collections in this group. One is the **jHWaves** file, which is for applications where the wave frequency is real. The other is the **jHWavesC** file, which is for applications with a complex frequency (i.e., frictionally damped waves or unstable waves). Using the two collections is very similar, with the main difference (to the user) is that with jHWavesC, you have to provide a complex number as the starting guess.

Examples of what your Julia screen will look like as you run the different codes are shown in section 2. The main routine, "jHWavesM" (or jHWavesCM for complex frequency), requires an input tuple that can be generated by "jHWavesSetup" (or jHWavesCSetup)and modified by "jHWavesFinch" (or jHWavesCFinch). When you run these functions, you simply need to answer the questions posed. Do not forget to use square brackets when entering more than one number on a line. (Much of this document is oriented to the real-frequency code, since the two cases are so similar.)

When running "jHWavesM" (or jHWavesCM), you will first be shown a few simple parameter values. You also get a section plot showing topography, stratification (minus a constant background value), and the mean flow. If you are allowing pauses during the run, you are advised of pauses as they happen. At this point, the alongshore wavenumber is displayed, and the program displays the frequency and the inverse resonance parameter (rrr = inverse of the integral of $|p|^2$ over x and z) for each ω estimate. When you are at a resonance (free mode), rrr becomes very small. You can thus watch the output converge onto a free modal frequency. Convergence to your given tolerance is then announced, and the free mode pressure is plotted as a section. A 10 second pause may be (your choice) announced at this point. You then also get energy diagnostics, section plots of u, v, w, ρ, and line plots of p, v, u at the surface. If you set the program to run more than one frequency, it will then display the next wavenumber and go on from there, and will add to a dispersion curve plot.

You can convert a real-frequency input tuple to a complex-frequency tuple by using jHWavesRtoC (for use with jHWavesC) or convert the imaginary tuple into the real form using jHWavesCtoR (for use with jHWaves).

5. Some Notes on using the software:

The $x = 0$ and $x = xmax$ boundary conditions can either be closed (as at a coast), or open. When there is no bottom friction, the coastal wall can have any depth. If there is bottom friction (i.e., with $r \neq 0$ in jHWavesC), then any closed boundary must occur in shallow water, meaning that $h << fL/N$ where L is a representative horizontal length scale. See also Mitchum and Clarke (1986) for other considerations.

If the $x = 0$ or $x = xmax$ boundary is open, the bottom should be flat at the boundary. In the case of an open boundary condition, the water does not have to be shallow. Care should be taken to make sure that the offshore boundary is located far from the main modal structure's location of maximum value.

The numerical grid extends one point outside the x, z domain at each end (nn is the total number of x grid points, and mm the total number of θ grid points). I.e., there is also a grid point outside each boundary. Thus, for example, nn = 5 gives one interior grid point, 2 on the boundaries and 2 outside the domain.

In choosing a grid size, caution is advised. One should experiment with grid sizes in order to be confident of a good answer. I find that nn = 80-120, mm = 20-30 seems to work well in many cases. The number of operations or amount of memory needed goes something like nn*mm*mm. There is a test for hydrostatic consistency (the ratio computed is $h_x\Delta x/(h\Delta\theta)$), and results seem best if the ratio given is less than around 1-5. This ratio gives a measure of when you might be in trouble estimating the horizontal pressure gradient in sigma coordinates.

The mean alongshore flow is assumed to be Gaussian (with different onshore, offshore, up and down length scales, as well as a variable center position and depth). This provides a fair bit of flexibility. Once the mean velocity field is computed, the mean density structure is computed using the thermal wind equation. The density field can be left undisturbed at either the offshore or onshore end of the domain. If you want to use some other functional form for mean velocity, you need only change lines 39-78 of the function jHWavesVzCal. The derivatives of the mean flow and the density corrections are computed elsewhere numerically, so that these are the only lines that would need changing.

Results for pressure, velocity and density are plotted.

Although the coastal long-wave limit is well defined when the mean alongshore flow is non-zero, the resulting wave modes are generally not orthogonal. This means that you cannot compute the first order wave equations coefficients (a_n, b_n, c_n) when there is a mean flow. Thus, you are not allowed to call the long-wave normalization and coefficient routine jHWavesLong when a mean flow is present. If you make the long wave approximation, you will only be

allowed to calculate one point on the dispersion curve. For the jHWavesC package, the long wave limit is not allowed at all.

All pressure fields are normalized with the coastal long-wave norm (although this is only valid when there be no mean flow and waves are long). Thus, all other variables are scaled consistently with the long wave normalization (Brink, 1989), corrected for the possibility of a closed offshore boundary. Units are always consistent within c.g.s., i.e. cm/sec, dyne/cm^2, gm/cm^3.

The search can be very inefficient if you give it 0 as the starting guess for frequency. Better to give it a nonzero value that is about the right order of magnitude.

The search can sometime miss a frequency. If it is a good solution, the inverse resonance parameter in the output (rrr) should be several orders of magnitude lower than surrounding values. If this is not true, you may either have a bad solution, or you may only be finding the real part of a complex wavenumber (which means you should use jHWavesC.jl to see if the flow is stable). Using a smaller accuracy tolerance (e) will tend to eliminate false solutions.

For an accuracy estimate ("e" option in jHWavesFinch), I find that 0.0001 works out fairly well. This means accuracy to 0.01% of the absolute value of the initial frequency guess.

The bottom stress is assumed to be proportional to the bottom velocity. For the errors associated with this assumption in the presence of a mean flow, see Brink (1997).

There is always an option to save model results including the "dispersion curve" information (wavenumbers and frequencies) and a pressure modal structure as a ".jld2" file.

In jHWaves, in the absence of a mean flow, the perturbation bottom frictional decay time is computed for each frequency (Brink, 1990), allowing for the possibility of a closed offshore boundary and variable resistance coefficient. The *e*-folding time is only valid for weak bottom friction (small *r*). Computing this, and the long wave coupling terms, are the only reasons for providing *r* for jHWaves. In the jHWavesC system, a complex frequency is calculated, and damping is accounted for in the imaginary part of the frequency. The perturbation damping time is not computed.

The format for the input tuple differs slightly from the general case that allows complex frequency (jHWavesC). In order to convert the tuple from one form to the other conveniently, there are two translation functions, "jHWavesCtoR" and "jHWavesRtoC". These functions are provided in both the jHWaves and jHWavesC packages.

Given the presence of the near-inertial spurious mode, and given that the "open" boundary condition is only reasonable for subinertial frequencies, the model results should only be trusted for subinertial frequencies, $|\omega| < |f|$ (except in the long wave limit, where ω/l is what is actually important, and the idea of super- or sub-inertial does not make sense). If, in computing a dispersion curve, you start to obtain frequencies greater than the inertial, the superinertial results should be discarded.

All frequencies are in radians/sec and all wavenumbers are in radians/cm. Because radians are nondimensional, this could also be written as "1/sec" etc.

You can get spurious results if the domain is too large relative to the natural scale of the wave under consideration. For example, with a domain size xmax = 70 km, trying to find a flat-bottom internal Kelvin wave with internal Rossby radius = 3 km yields bad results. Doing the same calculation with xmax $\leq$ 20 km yields reasonable results. This problem arises because the arbitrary forcing used to create the resonance is applied near xmax/2. Trouble can arise when the desired wave has negligible amplitude at this distance offshore.

Once you include a sheared mean flow, it is possible to get a mean state that is gravitationally unstable ($N^2 < 0$) in places. In this case, the unstable area will be displayed by red pluses in the density contour plot, and you will not be allowed to proceed.

When the Ertel potential vorticity $q = -N^2 f^*$ changes sign relative to f, symmetric instability is possible (Hoskins, 1974). So, the routine "jHWavesRhoCal" tests to see if f^*/f is negative anywhere. If this happens, f^* is contoured in red on the same plot as mean density, and a message appears on your Julia screen. If f^*/f does not become negative, it is not contoured, and no message about it is given. Experience with the program suggests that modes calculated when f^* changes sign are of questionable merit. Structures are then often complex and have short scales near the region of negative f^*. It seems probable that if, in nature, a flow is symmetrically unstable, it will quickly mix to a neutral state with zero or positive f^*/f. So, even a "good" modal solution for a symmetrically unstable case is of questionable value.

6. Disclaimer:

Although considerable effort has been made to make sure this software is correct and easy to use, there is no guarantee of perfection. If errors are found, or if documentation could be improved, please contact kbrink@whoi.edu .

References:

Brink, K.H., 1982. A comparison of long coastal trapped wave theory with observations off Peru. *Journal of Physical Oceanography*, **12**(8), 897–913.

Brink, K.H., 1989: Energy conservation in coastal-trapped wave calculations. *J. Phys. Oceanogr.,* 19, 1011-1016.

Brink, K.H., 1990: On the damping of free coastal-trapped waves. *J. Phys. Oceanogr*., 20, 1219-1225.

Brink, K. H., 1997. Time-dependent motions and the nonlinear bottom Ekman layer. *Journal of Marine Research*, **55**(4), 613–631.

Brink, K. H., 2006. Coastal-trapped waves with finite bottom friction. *Dynamics of Atmospheres and Oceans*, **41**, 172-190.

Brink, K.H., 2023: *Physical Oceanography of the Continental Shelf*. Princeton University Press, 248 pp.

Dale, A.C., J.M. Huthnance, and T.J. Sherwin, 2001: Coastal-Trapped Waves and Tides at Near-Inertial Frequencies. , **31**, 2958–2970.

Furue, R. and Y. Tanaka, 2025: Numerical calculation of coastal trapped wave modes. Unpublished document. https://eartharxiv.org/repository/view/10924/

Mitchum, G.T. and A.J. Clarke, 1986: The frictional nearshore response to forcing by synoptic scale winds. J. Phys. Oceanogr., 16, 934-946.

Mooers, CNK, 1975a: Several effects of a baroclinic current on the cross-stream propagation of inertial-internal waves. *Geophys. Fluid Dyn.,* 6, 245-275.

Mooers, CNK, 1975b: Several effects of a baroclinic current on the three-dimensional propagation of inertial-internal waves. *Geophys. Fluid Dyn*., 6, 277-284.

Wang, D.-P. and CNK Mooers, 1976: Coastal-trapped waves in a continuously stratified ocean. *J. Phys. Oceanogr.,* 6: 853-863.

Dale, A.C., J.M. Huthnance, and T.J. Sherwin, 2001: Coastal-Trapped Waves and Tides at Near-Inertial Frequencies. , **31**, 2958–2970.

Gill, A.E. and E.H. Schumann, 1974: The generation of long shelf waves by the wind. J. Phys. Oceanogr., 4, 83-90.

Huthnance, J.M., 1975: On trapped waves over a continental shelf. J. Fluid Mech., 69. 689-704.

Pedlosky, J., 1979: Geophysical Fluid Dynamics, Springer-Verlag, New York, 624pp.

Power, S. B., J. H. Middleton, and R. H. J. Grimshaw, 1989: Frictionally modified continental shelf waves and the subinertial response to wind and deep-sea forcing. J. Phys. Oceanogr., 19, 1486–1506, https://doi.org/10.1175/1520-0485(1989)019,1486: FMCSWA.2.0.CO;2.